Project Report.

1. Introduction

A secondary school needs an offline Learning Management System for four roles: Administrator, Lecturer, Head of Department (HoD) and Student. The distinguishing feature is **moderation**: students may not see a mark until the HoD has reviewed, approved and released it. Late work is accepted for three days with a 10% penalty per day, then scores zero. Every mark change must be auditable (who, when, old value, new value, why). The system is a C++17 console or simple GUI application with SQLite, built using the Waterfall model with V-Model verification (Sommerville, 2016; Pressman and Maxim, 2019).

2. Project Management Plan (Phase 1)

2.1 Feasibility

Dimension	Finding
Technical	Feasible. A local C++/SQLite application meets every stated need.
Operational	Feasible if the interface stays simple (the Principal's requirement) and each role gets a short manual.
Economic	Feasible. It runs offline on existing old hardware. Payments, video, auto-grading, plagiarism detection and web or mobile interfaces are out of scope.
Legal	Feasible if passwords are hashed and every mark change is audited, as the IT Officer requires.
Schedule	Feasible within 7 weeks (28 Sep to 13 Nov 2026) if the critical path is protected.

Recommendation: proceed to requirements engineering, subject to security, requirements management and time management.

2.2 Work breakdown and schedule

The WBS has nine groups: planning, requirements, design, implementation, verification and validation (V&V), configuration management and quality assurance (CM/QA), deployment, maintenance and completion. The Gantt chart uses three review gates: G1 requirements review (7 Oct), G2 design review (16 Oct, implementation may not start before it) and G3 testing review (5 Nov). The critical path runs through feasibility, stakeholder analysis, SRS, use cases, architecture, class and sequence diagrams, implementation (skeleton, data layer, authentication, workflows, integration), system testing and the v1.0 release. Maintenance (CR1, v1.1) and the final report follow in week 7.

2.3 Risk register (summary)

ID	Risk	Exposure	Mitigation	Owner
R1	Unclear or conflicting requirements	High	Analyse conflicts early and get agreement	Project Manager
R2	C++ implementation	Very high	Incremental build, core features first	Developer

ID	Risk	Exposure	Mitigation	Owner
	overruns			
R3	Database design errors cause data loss	Medium	Constraints, transactions, database tests	Developer
R4	Password or mark exposure	Medium	Salted hashes, RBAC, parameterised SQL	Developer
R5	Schedule slippage	Very high	Gantt monitoring, critical path first	Project Manager
R6	Major defects found late	High	Test automatically from implementation onward	Tester
R7	Git or CI problems	Low	Set up CI early, small commits	Developer
R8	System hard to use	Medium	Simple UI, per-role manuals	Project Manager

The register must be re-assessed and updated at each gate. The final report needs the updated version in the appendix.

3. Requirements Engineering (Phase 2)

3.1 Stakeholders and conflicts

Stakeholders are the Principal (simplicity, controlled release), Lecturer (create assignments, extensions, grading, feedback), HoD (review and release marks), Student Representative (resubmission, written feedback) and IT Officer (offline, security, performance).

Conflict	Resolution
Principal vs Lecturer: no mark visible until HoD checks it, versus marks as soon as marked	Marks stay hidden until the HoD moderates, approves and releases them.
HoD vs Student Rep: 10% per day penalty, versus late work must not be rejected	Accept late work for up to 3 days with 10% per day; zero after that.
Lecturer vs HoD: immediate marks, versus HoD sees every mark	The lecturer completes marking. The HoD must moderate before the student sees anything.

Eight ambiguities were also logged, each with a clarification: what "simple" means, who grants an extension, partial-day handling, whether day three is penalised or zero, how many resubmissions are allowed, what "quickly" means, whether 800 means registered or concurrent users, and what "protected" means for passwords.

3.2 Software Requirements Specification

The SRS contains 14 functional requirements (FR-01 to FR-14) and 7 non-functional requirements (NFR-01 to NFR-07), each prioritised and given a verification method. The functional requirements cover login, role restriction, administration of users, departments and courses, assignment creation, extensions, marking and feedback, student submission and confirmation, and mark-change recording (FR-13). The measurable non-functional requirements are:

- **NFR-01:** login completes within 2 seconds with 800 student records.

- **NFR-02:** normal operation without internet.
- **NFR-03:** no plaintext passwords.
- **NFR-04:** invalid input never terminates the program.
- **NFR-05:** 100% of mark changes are audit-logged.
- **NFR-06:** role-based access control (RBAC) enforced.
- **NFR-07:** tests run automatically in CI.

3.3 Use case model

The use case diagram covers the four actors. Login is included by all role actions, Resubmit Assignment and Handle Late Submission extend their parent use cases, and HoD functions cover Review Moderation, Approve Grades, Release Grades and department reports. The brief also requires three fully dressed use case specifications (preconditions, main, alternative and exception flows, postconditions, business rules), one of which must be "Submit assignment".

4. System Design (Phase 3)

4.1 Architecture

The chosen style is a **four-layer architecture**: Presentation, Business Logic, Data Access (Repository/DAO) and Persistence (SQLite). Rules such as late penalties and moderation live in services and the domain, so they are unit-testable, and all SQL stays in one layer. Two alternatives were rejected:

- **Model-View-Controller:** with a single console view, controllers tend to absorb business rules, which hurts testability and coverage. The trade-off accepted is some forwarding code between layers.
- **Client-server with a network database:** it contradicts the offline requirement and adds deployment and security overhead.

The component diagram traces to requirements: NFR-02 (offline), FR-02, FR-03 and NFR-06 (role enforcement), and FR-13 and NFR-05 (audit).

4.2 Class design, patterns and SOLID

An abstract `User` has four subclasses. Four patterns are used:

Pattern	Where	Why
Factory	`UserFactory`	Central creation of the right user subclass. New roles are easy to add.
Strategy	`LatePenaltyPolicy` (`ThreeDayGraduatedLatePenaltyPolicy`)	Penalty rules can change without touching the grading code.
Repository/DAO	`IUserRepository`, `ICourseRepository`, `ISubmissionRepository` and SQLite implementations	Business logic is independent of SQLite, and mock repositories enable unit tests.
Observer	Audit logging on mark changes	Every mark change produces an `AuditRecord`.

SOLID is addressed as follows: single responsibility (separate grading, audit and repository classes), open/closed (new policies as new classes), Liskov (all roles substitute for `User`), interface segregation (one repository interface per entity) and dependency inversion (services depend on `LatePenaltyPolicy` and repository interfaces).

4.3 Behavioural models

The **submission life cycle** is Available, In Progress, Submitted, (Late or Late with extension, or Zero), Marked, Moderated, Released. The HoD can return a mark for correction. Resubmission is allowed before the deadline or with permission, and each resubmission creates a new version. The **swimlane activity diagram** runs from student submission through lecturer marking and HoD moderation to release and the student's view. **Sequence diagrams** cover "Submit assignment" (on time, late, not permitted, resubmission) and "Moderate marks" (queue, moderate, release, student view).

4.4 Database design

The schema (Departments, Users, Courses, Enrollments, Assignments, Submissions, AuditLogs) is in 3NF. The `User` hierarchy is mapped as single-table inheritance with a `role` column. Collections such as a course's assignments become foreign keys. `AuditLogs.performed_by` uses `ON DELETE RESTRICT` so audit history cannot be orphaned. All SQL is parameterised.

5. Consistency Issues to Fix Before Implementation

The brief penalises artefacts that disagree with each other, so these should be resolved first:

1. **Missing schema elements.** The SQL has no table linking lecturers to courses (FR-06), no extensions table (FR-08, and CR1 needs class-wide extensions), no feedback column (FR-09), and no version or file path on `Submissions`, although resubmission versions appear in the design.
2. **Status values.** `ReturnedForCorrection` appears in the class and activity diagrams but not in the SQL `CHECK` list.
3. **ERD vs class diagram vs SQL.** The ERD shows a `Student` entity with course fields and a `Grade` entity that the SQL lacks, and it repeats `Submission` and audit boxes. The data dictionary uses `UserAccount` and `enrolled_at` while the SQL uses `Users` and different columns.
4. **Class diagram clean-up.** `Assignment` and `IUserRepository` are drawn twice. `Student` and `Lecturer` redeclare inherited attributes. `HeadOfDepartment` carries an odd `studentId`. Stray text such as "image_0.png" and the date notes in the state diagram should be removed.
5. **Requirement IDs and wording.** The traceability text says "NRF-05/06" (should be NFR). The SRS mixes "should" and "shall". Use "shall" consistently.
6. **Observer pattern.** It is described as "implicit". The brief requires the patterns to be implemented as shown, so either implement it explicitly or drop the claim.
7. **Penalty placement.** State where the penalty is applied (to the raw mark, at marking or at moderation) and keep the sequence diagram, the state machine and the code consistent.
8. **Use case scope.** Some use cases (Configure System Settings, Assign Roles, Manage Course Material, Generate Department Reports) have no matching requirement. Add requirements or remove them. Show generalisation, as the brief asks, and add the three dressed specifications.

6. Next Steps

Implement the layered design in C++17 (abstract `User`, Strategy and Repository, salted hashing with a recognised library, parameterised SQLite, exceptions and validation) with CMake and CI. Write the test plan with equivalence partitioning and boundary values on the 3-day penalty window, then automated tests with at least 70% coverage of the business layer, system tests from the use cases and a defect log. After that, add CM/QA evidence, the v1.0 release, CR1 (class-wide extensions, v1.1) with regression tests, and the professional, legal and ethical section. Throughout, keep the traceability matrix current.

7. Recommended Textbooks by Topic

Topic	Read
Process, V-Model, overall life cycle	Sommerville (2016) chapters on plans, requirements, design and testing; Pressman and Maxim (2019)
Planning, estimation, risk, scheduling	Hughes, Cotterell and Mall (2017)
Requirements, conflicts, measurable NFRs	Wiegers and Beatty (2013); ISO/IEC/IEEE 29148 (2018)
Use cases	Cockburn (2001)
UML diagrams	Fowler (2003); Larman (2004)
Patterns and architecture	Gamma et al. (1994); Fowler (2002); Martin (2017)
Database and normalisation	Elmasri and Navathe (2016)
C++17 and code quality	Stroustrup (2022); Meyers (2014); McConnell (2004)
Security and password storage	Anderson (2020)
Testing (partitioning, boundary values, V-Model levels)	Myers, Sandler and Badgett (2011); Spillner, Linz and Schaefer (2014)
Continuous integration	Humble and Farley (2010)

References

Anderson, R. (2020) *Security Engineering*. 3rd edn. Indianapolis: Wiley.

Cockburn, A. (2001) *Writing Effective Use Cases*. Boston: Addison-Wesley.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Harlow: Pearson.

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Fowler, M. (2003) *UML Distilled*. 3rd edn. Boston: Addison-Wesley.